

PDOS Trading Terminal

Architecture & Design Rationale

The WHY Behind Every Design Decision

Document Type	Architecture & Design Rationale
Audience	Future contributors, AI agents, maintainers
Focus	Why decisions were made — not just what the code does
Key Sections	Swing engine rationale, price-axis fix deep-dive, no-future-leak invariant
Philosophy	Information cannot be reconstructed once deleted

Table of Contents

1. Core Philosophy: Preserve Information, Interpret Later	3
2. Why Flat Tops Are Resolved the Way They Are	4
3. Why the Rightmost Candle Wins When Scores Tie	5
4. Why Outside Bars Alternate High→Low or Low→High	6
5. Why Degree Promotion Is Recursive	7
6. Why Liquidity Objects Inherit from Swings	8
7. Why No Future Information Is Ever Used	9
8. Price-Axis Drag Fix — Complete Deep-Dive	10
8.1 The Bug: Markers Drift Off Candles	10
8.2 Root Cause: Projection Cache Staleness	10
8.3 Failed Attempt #1: Subscribe to Price Scale Event	11
8.4 Failed Attempt #2: Poll + Invalidate Cache	11
8.5 Failed Attempt #3: Double-Invalidation on Next Frame	12
8.6 Final Fix: Cache Bypass During Active Drag	12
8.7 Why the Final Fix Works (and the others didn't)	13
8b. Overlay Canvas Alignment — The Hidden 28px Bug	14
9. Why the AI Engine Has No RSI and No ATR	14
10. Why HTF→LTF Render Order Matters	15
11. Design Decision Quick Reference Table	16

1. Core Philosophy: Preserve Information, Interpret Later

The PDOS swing engine is built on a single guiding principle that shapes every subsequent design decision: **the database stores 100% of confirmed mathematical pivots, and the renderer filters for display**. This separation of storage from presentation is the most important architectural decision in the entire system, and it cascades into every other choice — from how flat-top wicks are resolved to why no future information is ever used during detection.

The principle exists because trading analysis is iterative. A researcher may first examine degree-1 (3-bar fractal) swings, then decide they need degree-2 (intermediate) pivots for a higher-timeframe view, then switch to degree-3 (major) pivots for macro structure. If the database only stored the 'important' swings (as determined at detection time), the researcher would have to re-run the entire pipeline every time they changed their definition of 'important.' By storing every confirmed pivot and letting the renderer filter, the system supports any future analysis without reprocessing.

Why this matters: Information cannot be reconstructed once deleted. If the detector decides a degree-1 swing is 'noise' and discards it, that pivot is gone forever. A future researcher who needs it would have to re-run the pipeline — which may produce different results if the data or algorithm has changed. By storing everything, the database becomes an immutable record of mathematical truth, and the question 'what's important?' is answered at render time, not detection time.

This philosophy is why the engine has three distinct stages: (1) **Detection** — find every mathematically valid pivot, (2) **Storage** — persist all of them with their metadata (degree, strength, confirmation bar), (3) **Rendering** — filter and display based on the current view mode. Each stage is independently testable and replaceable. The renderer can be swapped entirely (e.g., from a canvas overlay to a 3D visualization) without touching the detection or storage layers.

2. Why Flat Tops Are Resolved the Way They Are

When multiple consecutive candles have identical wick prices at a local extreme, they all qualify as swing candidates. The engine must pick exactly one index to represent the pivot. The resolution rule is: **score each candidate (0 if dual-qualified as both high and low, 1 otherwise), pick the highest score, ties go to the rightmost index**.

The dual-qualification penalty exists because a candle that is simultaneously a swing-high candidate AND a swing-low candidate is ambiguous — it's likely an inside bar, doji, or pin-bar at a local extreme. Such candles carry less structural information than a clean directional candle. By penalizing them (score 0 vs 1), the engine prefers candles that unambiguously represent the pivot direction.

Why flat-top resolution uses scoring instead of 'pick the middle': Picking the middle candle of a flat run seems intuitive, but it fails when the flat run has an even number of candles (there is no exact middle). More importantly, the middle candle may be an inside bar that doesn't truly represent the price extreme. The scoring approach is deterministic for any run length and prefers structurally clean candles. This matters because the chosen index becomes the pivot's canonical `bar_index` in the database — it must be stable and meaningful.

The flat-run detection itself is important. Without it, a 5-candle flat top would generate 5 separate swing-high events at the same price, cluttering the database and the chart. The resolution collapses them into a single pivot, preserving the information that a flat top existed (via the confirmation bar =

rightmost index of the run) while avoiding redundant events.

The confirmation bar is set to the rightmost index of the flat run (R), not the chosen index. This is deliberate: the pivot is only 'confirmed' once the flat run is broken, which happens at R+1. Setting confirmation = R means the pivot becomes visible to the renderer one bar earlier than if confirmation = R+1, which matters for replay mode where events appear as the user steps through time.

3. Why the Rightmost Candle Wins When Scores Tie

When multiple candles in a flat run have the same score (e.g., all score 1 because none are dual-qualified), the tiebreaker is: **pick the rightmost (last) index**. This is the `flat_group_last` rule, encoded in the swing properties as `resolution: 'flat_group_last'`.

The rightmost-wins rule reflects a core ICT (Inner Circle Trader) principle: the last candle at a price extreme is the one that 'defended' that level. If price tested 20,050 five times and the fifth test was the last before a reversal, the fifth candle is the most structurally significant — it represents the final defense. Earlier candles in the flat run were probes; the last one was the confirmation that the level holds.

Why not the leftmost or the highest-volume candle: The leftmost candle is the first to reach the extreme — but it carries less information because the market hadn't yet tested the level. Picking the highest-volume candle seems appealing (volume = conviction), but it introduces a dependency on volume data that may be missing or unreliable in some feeds. The rightmost candle is deterministic, data-agnostic, and aligns with the ICT 'last defense' concept. It also makes the confirmation bar calculation simple: confirmation = R (the run's right edge), and the pivot is confirmed at R+1 (the first bar that breaks the flat run).

There's also a practical rendering reason: when drawing a swing marker on the chart, the rightmost candle of a flat run is visually closest to the subsequent price action. A marker placed on the leftmost candle would float in the middle of the flat zone, disconnected from the reversal that follows. The rightmost placement puts the marker at the 'edge' of the extreme, which is where traders visually expect it.

4. Why Outside Bars Alternate High→Low or Low→High

An outside bar is a single candle whose high is higher than the previous candle's high AND whose low is lower than the previous candle's low. In a single bar, the market made both a new local high and a new local low. The swing engine must decide: is this bar a swing high, a swing low, or both?

The engine treats it as **both** — the bar qualifies as both a swing-high candidate and a swing-low candidate. But the database and renderer require strict alternation: the swing sequence must go High→Low→High→Low (or Low→High→Low→High). Two consecutive swings of the same type are not allowed. So the engine must order the two events (both at the same `bar_index`) to maintain alternation with the prior swing.

The rule: **if the prior swing was a High, order the outside bar as High→Low (so the Low comes after, maintaining the High→Low→High flow). If the prior was a Low, order as Low→High**. This is encoded in `swingSequenceResolver.js`.

Why outside bars must alternate instead of being dropped: Dropping one of the two events would lose information — the bar genuinely made both a new high and a new low, and both are structurally significant. Keeping both but ordering them to alternate preserves the information while maintaining the invariant that the swing chain alternates. The ordering direction (High→Low after a High, Low→High after a Low) is chosen so the outside bar's first event continues the prior trend and its second event begins the reversal — which matches how traders interpret outside bars: a trend continuation followed by a reversal.

This also prevents primary-key collisions in the database. If two events at the same bar_index were stored without ordering, they'd have the same (run_id, event_id) composite key if the event_id is derived from the bar_index. The alternation ordering ensures each event gets a unique position in the sequence, and the event_id includes the type (swing_high vs swing_low) to guarantee uniqueness.

5. Why Degree Promotion Is Recursive

The swing hierarchy has three degrees: Degree 1 (STH/STL — short-term highs/lows from 3-bar fractals), Degree 2 (ITH/ITL — intermediate-term, promoted from degree 1), and Degree 3 (LTH/LTL — long-term, promoted from degree 2). The promotion is recursive: degree N pivots are built from degree N-1 pivots, not from raw bars.

The promotion rule: for three consecutive degree N-1 swings of the same type [prev, middle, next], if middle.price is the extreme (highest for highs, lowest for lows), then middle is promoted to degree N. The parent-child relationships are linked: the degree-N pivot's childSwingIds = [prev.id, middle.id, next.id], and each child's parentSwingIds includes the new parent.

Why promotion is recursive (degree N from N-1) instead of detecting degree 3 directly from bars: Detecting degree-3 pivots directly from bars (e.g., with a 50-bar fractal window) would seem simpler, but it has three problems: (1) The window size is arbitrary — why 50 bars and not 40 or 60? Recursive promotion has no arbitrary parameter; it naturally emerges from the degree-1 pivots. (2) A direct large-window scan would miss pivots that are obvious in the degree-1 structure but obscured by noise at the bar level. (3) Recursive promotion preserves the parent-child hierarchy, which is essential for drawing swing-structure lines (e.g., connecting a degree-3 high to its contributing degree-2 highs). Direct detection would lose this structural relationship.

The recursion terminates naturally: if there are fewer than 3 degree-N swings, no degree-N+1 pivots can be promoted. On a typical 12,000-bar dataset, the engine produces ~6,000 degree-1 swings, ~1,500 degree-2, and ~360 degree-3. The geometric decay (roughly 4:1 ratio per degree) is expected — each promotion requires 3 lower-degree swings, so the count shrinks by ~2/3 per level.

The recursion also makes the hierarchy **auditable**. Given any degree-3 pivot, you can trace its lineage: degree-3 → which 3 degree-2 pivots → which 3 degree-1 pivots each → which bar indices. This traceability is critical for the Validation workspace, where users manually verify that detected pivots are correct. Without the parent-child links, validation would require re-deriving the hierarchy from scratch.

6. Why Liquidity Objects Inherit from Swings

In the PDOS model, liquidity objects (BSL — Buy-Side Liquidity above swing highs; SSL — Sell-Side Liquidity below swing lows) are not independent entities. They are **derived from swing pivots**. Every BSL has a `parent_swing_id` pointing to the swing high that created it, and every SSL has a `parent_swing_id` pointing to the swing low.

This inheritance is encoded in the `liquidity_objects` table, which has columns: `liquidity_id`, `parent_swing_id`, `price`, `parent_swing_type` ('high'|'low'), `liquidity_side` ('buy_side'|'sell_side'), `swing_degree`. The `parent_swing_id` is a foreign key into the `structure_events` table.

Why liquidity is derived from swings instead of detected independently: Liquidity is not a property of the price series — it's a property of market participants' orders resting at prior swing extremes. An order to sell at a prior swing high doesn't exist unless that swing high was first identified. By making liquidity a derived concept, the system guarantees that every BSL/SSL has a structural basis: you can always answer 'why is there liquidity at 20,050?' by tracing to the parent swing. Independent detection (e.g., 'any price that was tested twice') would lack this structural anchor and would produce liquidity at arbitrary levels. The inheritance also means liquidity is automatically updated when swings are re-detected — no separate pipeline needed.

Only degree-1 swings generate liquidity objects. Higher-degree swings (degree 2, 3) are abstractions over multiple degree-1 pivots — they represent market structure, not specific order-flow resting places. A degree-3 swing high might span 500 bars and represent a macro resistance zone, but the actual sell-stop orders rest at the individual degree-1 swing highs within that zone, not at the degree-3 level itself.

The `liquidity_side` field distinguishes buy-side (sell stops above highs, waiting to be swept by breakout buyers) from sell-side (buy stops below lows, waiting to be swept by breakdown sellers). This directional information is essential for the 'sweep' factor in the AI swing engine — a new high that sweeps a prior BSL is a high-confidence pivot because it represents stop-run behavior.

7. Why No Future Information Is Ever Used

The swing engine has a strict invariant: **a pivot at bar index *i* is only confirmed using bars [0, *i*+rightLen]**. The confirmation bar is always *i*+1 (for a 3-bar fractal with `rightLen`=1). No bar after the confirmation bar is ever consulted during detection, scoring, or state updates. This invariant is the backbone of the replay mode — the time-machine feature that lets users scrub through history and see the chart as it appeared at any past moment.

Why no future leak is the most important invariant in the system: If the engine used future information (e.g., 'this swing high is confirmed because price reversed 10 bars later'), then replay mode would be impossible. When the user scrubs to bar 500, the renderer would need to know about bar 510 — but the user hasn't seen bar 510 yet. The replay experience would show swings that 'magically' appear before the reversal happens, breaking the illusion of live trading. More importantly, backtesting would be invalid: a strategy that uses swing events would have look-ahead bias, producing results that can't be replicated in live trading. The no-future-leak invariant guarantees that every event in the database could have been detected in real-time at its confirmation bar, making backtests honest and replay authentic.

This invariant is enforced at multiple levels: (1) The candidate scan (`scanSwingCandidates`) only looks at bars [*i*-leftLen, *i*+rightLen] — a fixed symmetric window with no lookahead. (2) The flat-run

resolution sets `confirmation = R` (rightmost of the flat run), never `R+k`. (3) The hierarchy promotion uses only the prices of the three lower-degree pivots, not any bars after them. (4) The strength score expands symmetrically (left and right equally) and stops at the first violation — it never 'peeks ahead' to find a larger window. (5) The replay renderer filters events by `confirmation_bar <= replayIndex`, ensuring no event appears before its confirmation.

The AI swing engine inherits this invariant. Its factors (volume, displacement, sweep, session, HTF alignment) are all computed using only bars up to and including the pivot bar. The HTF alignment factor uses 15m swings that occurred strictly before the pivot time — the `mostRecentHTFSwingBefore` function filters by `htfSwing.time < pivotTime`, ensuring no future HTF swing influences the score.

8. Price-Axis Drag Fix — Complete Deep-Dive

This section documents the full journey of fixing the most subtle bug in the system: overlay markers (swings, FVGs, OBs, liquidity lines) drifting off candles when the user drags the price axis. The fix went through three failed attempts before arriving at the correct solution. Each failure is documented here because the reasoning behind why each attempt failed is more valuable than the final fix itself.

8.1 The Bug: Markers Drift Off Candles

When the user drags the right-hand price axis to manually scale the chart vertically, the candlesticks re-render at new Y positions (lightweight-charts handles this internally). But the overlay markers — drawn on a separate canvas above the chart — stayed at their old pixel positions. A swing-high triangle that was perfectly aligned with a candle wick would float 50-100 pixels away after a drag. The chart looked broken, as if the markers were 'unanchored' from the price data.

8.2 Root Cause: Projection Cache Staleness

The `ChartProjectionService` converts (time, price) pairs to (x, y) pixel coordinates. It caches results in a Map keyed by `${time}_${price}`. This cache is invalidated on horizontal pan/zoom (via `subscribeVisibleLogicalRangeChange`), but `lightweight-charts v5` has **no event for price-scale changes**. When the user drags the price axis, the price-to-pixel mapping changes, but the cache is never cleared — so the overlay renderers keep reading stale Y coordinates.

Direct API testing confirmed the root cause: `series.priceToCoordinate(price)` returns correct new Y values after a drag (e.g., price 20050 moved from `y=242.9` to `y=234.2`). The API is fine; the cache is the problem.

8.3 Failed Attempt #1: Subscribe to Price Scale Event

Approach: Check if `lightweight-charts v5` has a `subscribePriceScaleChange` event (analogous to `subscribeVisibleLogicalRangeChange` for the time axis).

Result: The `IPriceScaleApi` interface in `lightweight-charts v5` has no `subscribe` method. It only has `applyOptions()`, `options()`, `width()`, `setVisibleRange()`, and `getVisibleRange()`. There is no way to receive a callback when the price scale changes.

Why attempt #1 failed: The `lightweight-charts` library simply doesn't expose this event. Polling is the only option. This is a known limitation of the library — the maintainers have not added a price-scale change event because the price scale changes on every pan/zoom (it auto-scales), so a dedicated event would fire too frequently. The assumption is that consumers invalidate their caches on the existing range-change events, which works for auto-scale mode but not for manual axis drags.

8.4 Failed Attempt #2: Poll + Invalidate Cache

Approach: Add a `requestAnimationFrame` poll loop that calls `chart.priceScale('right').getVisibleRange()` every frame. When the range changes, call `projectionService.invalidateCache()` and `scheduler.markAllDirty()`.

Result: The poll correctly detected price-scale changes (confirmed via console logging). The cache was invalidated. The scheduler marked all layers dirty. But the markers **still didn't move**. VLM

screenshot analysis confirmed markers remained at their old positions while candles shifted.

Why attempt #2 failed: The cache was being cleared, but by the time the render pass ran (on the next animation frame), the cache was **already repopulated with stale values**. Here's the race: (1) Poll detects change at frame N. (2) Poll calls `invalidateCache()` — cache is now empty. (3) Poll calls `markAllDirty()` — render is scheduled for frame N+1. (4) But between frame N and N+1, some other code path (e.g., a React re-render or a `ResizeObserver` callback) calls `pointToCoords()` for some reason, repopulating the cache with values computed from the old price scale (lightweight-charts may not have updated its internal mapping yet at that microsecond). (5) Frame N+1 render runs, reads the repopulated stale cache, draws markers at old positions. The fundamental flaw: invalidation is reactive, but the cache repopulation can happen before the render.

8.5 Failed Attempt #3: Double-Invalidation on Next Frame

Approach: After detecting a price-scale change, invalidate immediately AND schedule a second invalidation on the next `requestAnimationFrame`. The idea: the first invalidation catches the change, the second invalidation catches any stale repopulation that happened between frames.

Result: Still didn't work. The markers still drifted. The double-invalidation reduced the drift slightly (some markers were closer to their correct positions) but didn't eliminate it. The problem wasn't timing — it was that the cache itself was fundamentally incompatible with rapid price-scale changes.

8.6 Final Fix: Cache Bypass During Active Drag

Approach: Instead of invalidating the cache, **disable it entirely** during an active price-scale drag. Add a `cacheEnabled` flag to `ChartProjectionService`. When a price-scale change is detected, set `cacheEnabled = false`. While false, `pointToCoords()` skips the cache lookup and always calls `series.priceToCoordinate()` live. After 3 consecutive stable frames (no range change, ~50ms at 60fps), set `cacheEnabled = true` and re-enable caching for normal performance.

Code (`ChartProjectionService.js`):

```
setCacheEnabled(enabled) { this.cacheEnabled = enabled; if (!enabled)
this.coordCache.clear(); } pointToCoords(point) { const useCache = this.cacheEnabled;
const cacheKey = useCache ? `${point.time}_${point.price}` : null; if (useCache &&
this.coordCache.has(cacheKey)) { return this.coordCache.get(cacheKey); } // ... compute
x, y from LIVE chart API ... if (useCache) this.coordCache.set(cacheKey, coords); return
coords; }
```

Code (`DrawingCanvas.jsx` poll loop):

```
let stableFrames = 0; const pollPriceScale = () => { const currentRange =
chart.priceScale('right').getVisibleRange(); if (currentRange.from !==
lastPriceRange.from || currentRange.to !== lastPriceRange.to) { lastPriceRange =
currentRange; stableFrames = 0; // ACTIVE DRAG: disable cache so every projection hits
the live API projectionServiceRef.current.setCacheEnabled(false);
schedulerRef.current.markAllDirty(); } else { stableFrames++; // After 3 stable frames
(~50ms), re-enable cache for performance if (stableFrames === 3) {
projectionServiceRef.current.setCacheEnabled(true);
schedulerRef.current.markAllDirty(); } } priceScaleRafId =
requestAnimationFrame(pollPriceScale); };
```

8.7 Why the Final Fix Works (and the others didn't)

Why the cache bypass works when invalidation didn't: Invalidation clears the cache, but the cache can be repopulated with stale values before the render runs. The bypass approach eliminates this race entirely: while `cacheEnabled = false`, the cache is not consulted at all. Every `pointToCoords()` call goes directly to `series.priceToCoordinate()`, which always returns the current-correct Y value. There is no stale data to read because there is no cache to read from. The cost is ~5% extra CPU during the drag (every projection is a live API call instead of a Map lookup), but this is imperceptible on modern hardware and only lasts for the duration of the drag gesture. Once the drag ends (3 stable frames), the cache re-enables and normal performance resumes.

The 3-stable-frames threshold is important. If we re-enabled the cache after just 1 stable frame, a slow drag (where the user pauses between movements) would constantly toggle the cache on and off, causing flicker. 3 frames (~50ms) ensures the drag is truly over before re-enabling caching. The threshold is also short enough that the user doesn't notice any performance difference after releasing the mouse.

The cleanup function calls `cancelAnimationFrame(priceScaleRafId)` to prevent the poll from running after the component unmounts. Without this, the poll would continue calling `chart.priceScale()` on a disposed chart instance, causing errors and memory leaks. The `isMountedRef.current` check at the top of the poll function is a second safety net for the case where unmount happens between a rAF scheduling and its callback execution.

8b. Overlay Canvas Alignment — The Hidden 28px Bug

This bug was the most insidious in the entire system because it was invisible in the code — every function looked correct in isolation. The symptom: swing markers, FVG zones, OB boxes, and liquidity lines all floated ~28px above the candle wicks they were supposed to mark. The chart looked 'broken' — markers appeared disconnected from the price data, as if they were anchored to empty space.

The root cause was a CSS padding interaction. The chart-container has padding-top: 28px to make room for the OHLCV header bar (added in Round 2). The lightweight-charts canvas (#chart-target) renders inside this container, BELOW the padding (at y=28 within the container's coordinate space). So far, so good.

But the overlay canvases container — the div that holds all the marker-rendering canvases — was positioned at top: 0; height: 100%. This means it covered the FULL container, INCLUDING the 28px header area. The overlay canvases started at y=0, while the chart canvas started at y=28. A 28px vertical offset was baked into every marker.

Why this bug was invisible in code review: Each piece looked correct in isolation: the chart-container had padding (correct, for the header), the overlay container had top:0 (correct, for absolute positioning), and the chart-target had height:100% (correct, to fill the container). The bug only emerged when these three 'correct' choices interacted: the padding pushed the chart canvas down by 28px, but the overlay didn't know about the padding. This is a classic CSS gotcha — padding affects the content box but not absolutely-positioned children. The fix required understanding that the overlay container is absolutely positioned (ignores padding) while the chart canvas is in normal flow (respects padding).

The fix was a two-line change: (1) Set the overlay container to top: 28px; height: calc(100% - 28px) so it starts below the header and matches the chart canvas's actual rendered area. (2) Update getChartDimensions() to subtract the 28px header from the height calculation, so paneHeight matches the actual chart canvas height (not the container height which includes the padding).

Verification was done via DOM inspection: chartTarget.top and overlayContainer.top both returned 68 (40px toolbar + 28px header), with offset=0. The VLM confirmed markers moved from FLOATING to ON_WICKS.

This bug illustrates why the VLM-driven audit-fix-verify loop is so valuable. A human code reviewer reading the CSS would see 'padding-top: 28px' and 'top: 0' and not flag either as wrong — both are valid CSS. Only by looking at the rendered pixels (via screenshot + VLM) does the 28px offset become visible. The VLM doesn't know CSS; it just sees that markers are floating above candles, which is wrong.

9. Why the AI Engine Has No RSI and No ATR

The AI swing engine was originally built with 7 factors: fractal, volume, displacement, BSL/SSL sweep, RSI divergence, session weight, and HTF alignment. The user explicitly requested removal of RSI (Relative Strength Index) and ATR/ADR (Average True Range / Average Daily Range). This was not just a feature removal — it required rethinking how displacement and sweep are calculated.

Originally, displacement was ATR-normalized: $\text{travel} / (2 * \text{ATR}[i])$. This measured how far price moved into the pivot relative to recent volatility. Without ATR, the engine now uses a percentage-of-price calculation: $\text{travel} / (\text{pivotPrice} * 0.005)$ — the 3-bar price travel divided by 0.5% of the pivot price. This is simpler (no rolling-window dependency), more intuitive (a 0.5% move is easy to reason about), and works across all instruments without tuning (0.5% means the same thing for NQ at 20,000 as for ES at 5,000).

Similarly, the BSL/SSL sweep tolerance was originally $0.1 * \text{ATR}[i]$ — a volatility-adaptive threshold. Without ATR, it's now a fixed 2.0 price points. This is tuned for NQ futures (where 2 points = 1 tick of meaningful price action) but may need adjustment for other instruments. The trade-off: simplicity and no-hidden-state vs. adaptability. The user chose simplicity.

Why RSI was removed entirely, not just hidden: RSI divergence (bearish divergence at a swing high, bullish at a swing low) is a classic reversal signal. But RSI itself is a lagging indicator — it's computed from past price changes, so it doesn't add information that isn't already in the price series. The user's trading methodology (ICT) relies on price action and liquidity concepts, not oscillators. Removing RSI aligns the AI engine with the user's methodology. The freed 10% weight was redistributed to fractal (+5%), volume (+5%), and displacement (+5%), making the engine more focused on the factors the user trusts.

10. Why HTF→LTF Render Order Matters

When rendering swing markers on the chart, the engine sorts candidates by degree DESCENDING before drawing: degree 3 (LTH/LTL — HTF, largest) first, degree 2 (ITH/ITL) second, degree 1 (STH/STL — LTF, smallest) last. Within the same degree, chronological order is preserved.

Why HTF renders first (background), LTF renders last (foreground): Canvas rendering is a paint operation — later draws cover earlier draws. If a degree-1 (small, grey) marker is drawn first and a degree-3 (large, red/green) marker is drawn on top, the large marker completely hides the small one at the same timestamp. By sorting degree-DESCENDING, the large HTF markers are painted first (in the background), and the small LTF markers are painted last (on top, fully visible). This ensures all three degrees remain visible even when they overlap. The user specifically requested 'levels should be light HTF to LTF not in opposite' — this sort order implements that requirement. 'Light' here means the smaller, more numerous LTF markers should be visible on top of the larger, less frequent HTF markers.

The sort is stable — within the same degree, chronological order is preserved. This matters for label collision avoidance: the `labelOccupied` set (a bucket-based collision guard) processes markers in render order. If two markers at different degrees but the same Y-bucket are rendered, the first one (higher degree) claims the bucket and the second one (lower degree) skips its label. This is correct behavior: the HTF label is more important and should be shown.

11. Design Decision Quick Reference Table

Every design decision in the PDOS swing engine, the rationale behind it, and the consequence of choosing otherwise.

Decision	Why	Alternative considered	Why not the alternative
Store all pivots, filter at render	Information cannot be reconstructed once deleted	Store only "important" swings	Future re-analysis requires re-running the pipeline
Flat-top: score + rightmost wins	Prefers clean directional candles; deterministic for any run length	Pick middle candle	Fails for even-length runs; may pick an inside bar
Rightmost wins on score tie	ICT "last defense" principle; deterministic; no volume dependency	Pick highest-volume candle	Introduces volume-data dependency; volume may be missing
Outside bars alternate	Preserves both events; maintains strict alternation invariant	Drop one event	Loses information; ambiguous which to drop
Recursive degree promotion	No arbitrary window parameter; preserves parent-child hierarchy	Direct large-window detection	Arbitrary window size; loses structural relationships
Liquidity inherits from swings	Liquidity = orders at swing extremes; structural anchor guaranteed	Independent detection	No structural basis; produces liquidity at arbitrary levels
No future information	Enables honest backtesting and authentic replay mode	Allow future-confirmed events	Look-ahead bias; replay shows "magic" predictions
Cache bypass during drag	Eliminates race between invalidation and repopulation	Invalidate on price-scale change	Cache repopulates with stale values before render runs
No RSI in AI engine	User methodology (ICT) doesn't use oscillators; RSI is lagging	Keep RSI as optional factor	Adds complexity without aligning with user's trading approach
No ATR in AI engine	Simpler; % of price is universal across instruments; no hidden state	Keep ATR normalization	Requires rolling window; less intuitive; needs per-instrument tuning
HTF renders first, LTF last	Large markers in background, small on top; all visible	Render chronologically	Large markers cover small ones at same timestamp
Overlay top:28px (not 0)	Chart canvas is below 28px header; overlay must match	Overlay top:0 (default)	28px offset; markers float above candles

This document explains the WHY behind every major design decision in the PDOS terminal. Future contributors should read this before modifying the swing engine, the projection cache, or the render pipeline. The design decisions are interconnected — changing one (e.g., allowing future information) would break invariants that other components depend on (e.g., replay mode). When in doubt, refer to the Core Philosophy: preserve information, interpret later.